

PRÁCTICA N.º 6 — ENUNCIADO

Mensajería asíncrona con RabbitMQ

Eventos, exchanges y colas: el cierre del bloque de comunicación, donde el que llama deja de esperar respuesta

Asignatura	COM-600 · Microservicios · Gestión 2/2026
Tema del programa	Tema 4 — Comunicación entre Microservicios
Modalidad	Parte 1 guiada en laboratorio · Parte 2 individual
Duración estimada	Parte 1: 4 horas · Parte 2: trabajo autónomo
Puntaje	100 puntos (40 + 60)
Carpeta de trabajo	practica6-rabbitmq

Nota: Este documento es el ENUNCIADO: contiene los comandos, las explicaciones y los ejercicios. Las capturas y las respuestas NO se escriben aquí, sino en el documento complementario «Hoja de Evidencias», que es el que se entrega.

1. Objetivos

Al finalizar esta práctica el estudiante será capaz de:

- Reconocer la diferencia práctica entre llamar a un servicio y publicar un evento, después de tres prácticas de comunicación síncrona.
- Levantar un broker RabbitMQ con Docker Compose y operar su consola de administración.
- Distinguir exchange, cola, binding y clave de enrutamiento, y crearlos primero a mano para verlos antes de programarlos.
- Publicar un evento de dominio desde un microservicio en el momento en que una operación se confirma.
- Consumir ese evento desde un segundo microservicio y demostrar el desacoplamiento apagando el consumidor.
- Enrutar un mismo evento a varias colas con exchanges directo y temático, sin tocar el código del productor.
- Garantizar que un mensaje no se pierda: confirmación manual, colas durables y mensajes persistentes.
- Configurar reintentos limitados y una cola de mensajes muertos para el mensaje que nunca se puede procesar.
- Escribir un consumidor idempotente y demostrar que una entrega repetida no produce un efecto doble.

2. Qué necesita saber antes de empezar

El contenido conceptual de esta práctica se desarrolla en **Tema 4 — Comunicación entre Microservicios**. Aquí solo se recuerda lo mínimo necesario para ejecutar los laboratorios.

- **Exchange:** la puerta de entrada del broker. El productor siempre publica aquí, nunca directamente en una cola.
- **Cola:** el depósito donde el mensaje espera hasta que un consumidor lo tome. Si nadie consume, se acumula.
- **Binding y clave de enrutamiento:** el binding es la regla que une un exchange con una cola; la clave de enrutamiento es la etiqueta del mensaje que esa regla compara.
- **Ack:** la confirmación que el consumidor envía cuando terminó de procesar. Recién entonces el broker borra el mensaje.
- **Cola de mensajes muertos:** la cola donde termina el mensaje que fue rechazado, expiró o agotó sus reintentos. Es la bandeja de los casos que un humano debe mirar.

Repaso: Hasta la Práctica 5 toda la comunicación fue síncrona: REST en la Práctica 3, GraphQL en la 4 y gRPC en la 5. En las tres, quien llamaba se quedaba esperando la respuesta y heredaba la falla del otro servicio. Esta práctica cierra el bloque de comunicación introduciendo el desacoplamiento por eventos: el productor publica y sigue. La discusión conceptual —estilos de comunicación, acoplamiento temporal, entrega al menos una vez, coreografía frente a orquestación— está en las diapositivas del Tema 4. Aquí se ejecuta.

3. Requisitos previos

Requisito	Verificación	Observación
Docker Desktop o Engine 24+	<code>docker --version</code>	Con el complemento compose: <code>docker compose version</code> debe responder
Node.js 20 o superior	<code>node --version</code>	La 20 o la 22 LTS
npm 10 o superior	<code>npm --version</code>	Viene con Node
curl o Postman	<code>curl --version</code>	En Windows use Git Bash: trae curl y permite copiar los comandos tal cual
Navegador web	Cualquiera actualizado	La consola de administración del broker se abre en el navegador, en el puerto 15672
Puertos 5672 y 15672 libres	<code>docker ps</code>	Si otro contenedor los ocupa, deténgalo antes de empezar
Git	<code>git --version</code>	Para subir la entrega al repositorio

Atención: Cree una carpeta de trabajo llamada `practica6-rabbitmq` y realice TODO el trabajo dentro de ella. Esa carpeta se sube al repositorio al final.

Los comandos se muestran precedidos por el símbolo `$`. NO escriba el símbolo `$` al copiar el comando.

Esta práctica necesita hasta cuatro terminales abiertas a la vez: el broker, el productor, el consumidor y una para las pruebas con curl. Ordénelas antes de empezar.

No borre el contenedor del broker entre laboratorios. Varios pasos comparan el estado de las colas antes y después de un reinicio.

4. Estructura y forma de entrega

La práctica se apoya en dos documentos:

Documento	Para qué sirve	¿Se entrega?
Enunciado (este archivo)	Guía de trabajo: comandos, explicaciones, ejercicios y rúbrica. Se consulta mientras se trabaja.	No
Hoja de Evidencias	Formulario donde se pegan las 20 capturas y se responden las 8 preguntas de comprobación de la Parte 1.	Sí, en PDF

El trabajo tiene dos partes obligatorias:

Parte	Contenido	Puntaje
PARTE 1 — Guiada	8 laboratorios paso a paso. Se ejecutan los comandos y se documenta cada resultado en la Hoja de Evidencias.	40 pts
PARTE 2 — Ejercicios propuestos	6 ejercicios de resolución individual. Se entregan como código en el repositorio más un informe en PDF.	60 pts
TOTAL		100 pts

Entregables

- Hoja de Evidencias en PDF, con las capturas pegadas y las preguntas de comprobación respondidas (Parte 1).
- Repositorio Git público con la carpeta practica6-rabbitmq: docker-compose.yml, el microservicio productor, el consumidor, los scripts de topología y un README.md que explique cómo levantar todo desde cero.
- Informe PDF de la Parte 2 con la resolución de cada ejercicio, las capturas de la consola de administración que demuestren cada comportamiento y las respuestas a lo que se pide argumentar.

PARTE 1 — PRÁCTICA GUIADA

Ejecute cada comando en el orden indicado. Cuando aparezca un aviso de «Captura N.º», tome la captura de pantalla y péguela en el recuadro correspondiente de la Hoja de Evidencias. Las «Pregunta N.º» se responden en esa misma hoja, con sus propias palabras: son las que demuestran que entendió lo que ejecutó.

Consejo: Trabaje con los dos documentos abiertos a la vez: este enunciado para leer y ejecutar, y la Hoja de Evidencias para ir registrando. Las capturas y las preguntas están numeradas igual en ambos.

Laboratorio 0 — El broker en marcha y su consola de administración

En las prácticas 3, 4 y 5 solo había dos actores: quien llamaba y quien respondía. Hoy aparece un tercero en el medio, el broker, y lo primero es verlo funcionando.

Paso 1. Verifique las versiones instaladas

Abra una terminal en su carpeta de trabajo y ejecute:

```
$ docker --version
$ docker compose version
$ node --version
$ curl --version
$ docker ps
```

El último comando debe responder con una tabla, aunque esté vacía. Si responde con un error de conexión, Docker no está iniciado.

Captura N.º 1: salida de los cinco comandos anteriores en su terminal. → *péguela en la Hoja de Evidencias.*

Paso 2. Escriba el docker-compose.yml del broker

Cree la carpeta practica6-rabbitmq y dentro el archivo docker-compose.yml. La imagen elegida termina en -management: es la que trae la consola de administración, y sin ella esta práctica sería invisible.

```
practica6-rabbitmq/docker-compose.yml
services:
  rabbitmq:
    image: rabbitmq:3.13-management
    container_name: com600-rabbit
    hostname: rabbit-uno
    ports:
      - "5672:5672"      # AMQP: por aquí hablan los microservicios
      - "15672:15672"    # consola de administracion, en el navegador
    environment:
      RABBITMQ_DEFAULT_USER: admin
      RABBITMQ_DEFAULT_PASS: admin123
    volumes:
```

```
- rabbit_datos:/var/lib/rabbitmq  
  
volumes:  
  rabbit_datos:
```

Sobre el usuario: El usuario guest que trae la imagen por omisión solo puede conectarse desde la propia máquina del broker. Creamos admin para que mañana, cuando el productor viva en otro contenedor, nada cambie. La contraseña de esta práctica es de laboratorio: en un despliegue real va en un secreto, nunca en el archivo del repositorio.

Paso 3. Levante el broker y espere a que termine de arrancar

RabbitMQ tarda unos quince segundos en estar listo. Si conecta antes, obtendrá un error de conexión que no es culpa de su código.

```
$ docker compose up -d  
$ docker compose ps  
$ docker compose logs rabbitmq | tail -5  
# espere hasta ver la línea: Server startup complete
```

Captura N.º 2: salida de docker compose ps con el contenedor com600-rabbit en estado running y las últimas líneas del registro con «Server startup complete». → *péguela en la Hoja de Evidencias.*

Paso 4. Entre a la consola de administración

Abra en el navegador <http://localhost:15672> e ingrese con admin / admin123. Recorra las pestañas Overview, Connections, Exchanges y Queues. Todavía no hay nada suyo: los exchanges que aparecen con el prefijo amq. son los que el broker crea siempre.

Esta consola será su instrumento de medición durante toda la práctica. Cuando algo no funcione, la respuesta casi siempre está aquí antes que en la terminal.

Captura N.º 3: consola de administración de RabbitMQ abierta en la pestaña Overview, con el nombre del nodo rabbit-uno visible. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 1: En las prácticas 3, 4 y 5 el cliente hablaba directamente con el servicio y esperaba la respuesta. Aquí aparece un tercer proceso en el medio. ¿Qué gana el sistema al introducirlo y qué problema nuevo se crea con él? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

El broker está corriendo y usted puede verlo por dentro. A partir de aquí, todo lo que ocurra tendrá dos evidencias: la terminal y la consola.

Laboratorio 1 — Anatomía del enrutamiento: exchange, cola y binding

Antes de escribir una sola línea de código, vamos a construir a mano el camino que recorrerá un mensaje. Crear estas tres piezas desde la consola tiene un objetivo: verlas. Después el código hará exactamente lo mismo, pero ya sin misterio.

Paso 1. Cree el exchange

En la consola vaya a Exchanges → Add a new exchange. Complete: Name = inscripciones, Type = direct, Durability = Durable. Deje el resto como está y pulse Add exchange.

Un exchange no almacena nada. Es una oficina de clasificación: recibe el mensaje, mira su clave de enrutamiento y decide a qué colas copiarlo. Si no hay ninguna cola enlazada que coincida, el mensaje se descarta en silencio.

Captura N.º 4: lista de exchanges de la consola con el exchange inscripciones recién creado, mostrando su tipo direct y la marca D de durable. → *péguela en la Hoja de Evidencias.*

Paso 2. Cree la cola

Vaya a Queues and Streams → Add a new queue. Complete: Type = Classic, Name = notificaciones.correo, Durability = Durable. Pulse Add queue.

La cola sí almacena. Fíjese en las columnas Ready (mensajes esperando), Unacked (entregados pero todavía sin confirmar) y Total. Esas tres cifras cuentan casi toda la historia de un sistema de mensajería.

Paso 3. Enlace la cola al exchange con una clave de enrutamiento

Entre al exchange inscripciones, abra la sección Bindings y agregue: To queue = notificaciones.correo, Routing key = inscripcion.confirmada. Pulse Bind.

Acaba de declarar la regla completa: «todo lo que llegue al exchange inscripciones con la etiqueta inscripcion.confirmada, cópielo a la cola notificaciones.correo».

Captura N.º 5: página del exchange inscripciones mostrando el binding hacia la cola notificaciones.correo con la clave inscripcion.confirmada. → *péguela en la Hoja de Evidencias.*

Paso 4. Publique un mensaje a mano y recójalo

En la misma página del exchange, abra Publish message. En Routing key escriba inscripcion.confirmada y en Payload escriba {"prueba":1}. Pulse Publish message. Vaya luego a Queues → notificaciones.correo → Get messages, ponga Ack Mode en «Nack message requeue true» y pulse Get Message(s).

Repita la publicación con la clave inscripcion.anulada. El mensaje se pierde: no existe ningún binding que coincida y el exchange no tiene dónde dejarlo. Compruébelo mirando el contador Ready de la cola, que no sube.

Cuidado: El modo de recogida importa: si elige «Automatic ack» al hacer Get messages, la consola confirma el mensaje y este desaparece de la cola. Es la primera forma, y la más frecuente, de perder un mensaje sin darse cuenta.

Captura N.º 6: ventana Get messages mostrando el contenido del mensaje recuperado, con su clave de enrutamiento y sus propiedades. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 2: El productor publica en el exchange y nunca nombra a la cola. Si mañana el área de finanzas también necesita enterarse de cada inscripción confirmada, ¿qué hay que modificar en el productor? Justifique su respuesta con lo que acaba de hacer a mano. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Estas tres piezas —exchange, cola y binding— son todo el mecanismo. El resto de la práctica solo las combina de maneras distintas.

Laboratorio 2 — El productor: publicar un evento cuando la operación se confirma

El microservicio de Inscripciones registra al estudiante y, en el mismo instante en que la operación queda confirmada, anuncia el hecho. No pide nada a nadie: informa.

Paso 1. Prepare el proyecto del productor

```
$ mkdir -p practica6-rabbitmq/inscripciones
$ cd practica6-rabbitmq/inscripciones
$ npm init -y
$ npm install express amqplib dotenv
# amqplib es el cliente del protocolo AMQP para Node
```

Paso 2. Externalice la configuración

Cree el archivo .env dentro de la carpeta inscripciones. Ni la dirección del broker ni el nombre del exchange deben estar escritos dentro del código.

```
inscripciones/.env
RABBITMQ_URL=amqp://admin:admin123@localhost:5672
EXCHANGE=inscripciones
CLAVE=inscripcion.confirmada
PORT=3000
```

Paso 3. Escriba la conexión al broker

Cree el archivo cola.js. La conexión se abre una sola vez, al arrancar el servicio, y se reutiliza: abrir una conexión por cada mensaje es un error caro.

```
inscripciones/cola.js
const amqp = require("amqplib");
let canal;

async function conectar() {
  const conexion = await amqp.connect(process.env.RABBITMQ_URL);
  canal = await conexion.createChannel();
  const ex = process.env.EXCHANGE;
  await canal.assertExchange(ex, "direct", { durable: true });
  console.log("[inscripciones] conectado al broker");
  process.on("SIGINT", async () => {
    await canal.close(); await conexion.close(); process.exit(0);
  });
}
```

Por qué assertExchange: Asegurar el exchange desde el código no lo duplica: si ya existe con los mismos parámetros, no hace nada. Es lo que permite que el sistema se levante desde cero en una máquina limpia sin que nadie tenga que entrar a la consola a crear nada.

Paso 4. Escriba la función que publica

Agregue al mismo archivo la función de publicación y la exportación.

```
inscripciones/cola.js (agregar al final)
function publicar(clave, evento) {
  const cuerpo = Buffer.from(JSON.stringify(evento));
  // persistent: true pide que el mensaje se escriba en disco
  return canal.publish(process.env.EXCHANGE, clave, cuerpo, {
    persistent: true,
  });
}
```

```
    contentType: "application/json",  
  });  
}  
  
module.exports = { conectar, publicar };
```

Paso 5. Escriba la API que confirma la inscripción

Cree index.js. Observe el orden: primero se guarda el registro, y solo cuando la operación quedó confirmada se publica el evento. Publicar antes de confirmar es anunciar algo que todavía puede no ocurrir.

```
inscripciones/index.js  
require("dotenv").config();  
const express = require("express");  
const { conectar, publicar } = require("../cola");  
  
const app = express();  
app.use(express.json());  
const inscripciones = [];  
  
app.post("/inscripciones", (req, res) => {  
  const { estudiante, correo, curso } = req.body;  
  if (!estudiante || !correo || !curso) {  
    return res.status(400).json({ error: "faltan datos obligatorios" });  
  }  
  const registro = { id: "INS-" + Date.now(), estudiante, correo, curso };  
  inscripciones.push(registro);  
  publicar(process.env.CLAVE, { tipo: "InscripcionConfirmada", ...registro });  
  res.status(201).json(registro);  
});
```

Paso 6. Complete el arranque del servicio

```
inscripciones/index.js (agregar al final)  
app.get("/inscripciones", (_req, res) => res.json(inscripciones));  
  
conectar()  
  .then(() => app.listen(process.env.PORT, () =>  
    console.log("[inscripciones] API en el puerto " + process.env.PORT)))  
  .catch((e) => {  
    console.error("no se pudo conectar al broker:", e.message);  
    process.exit(1);  
  });
```

Si el broker no responde, el servicio no arranca. Es una decisión deliberada: un productor que arranca sin poder publicar acepta operaciones cuyos eventos se pierden.

Paso 7. Publique la primera inscripción

Deje el productor corriendo en una terminal y use otra para la prueba.

```
# terminal 2  
$ node index.js  
  
# terminal 3  
$ curl -i -X POST http://localhost:3000/inscripciones \  
-H "Content-Type: application/json" \  
-d '{"estudiante":"Ana Gutierrez","correo":"ana@usfx.bo","curso":"COM-600"}'
```

Captura N.º 7: las dos terminales: el productor conectado al broker y la respuesta 201 con el registro devuelto por curl. → *péguela en la Hoja de Evidencias.*

Paso 8. Mire dónde quedó el evento

Vuelva a la consola, entre a Queues and Streams → notificaciones.correo y observe la columna Ready.

Hay un mensaje esperando y todavía nadie lo ha leído. La API respondió 201 en milisegundos sin que exista un consumidor: eso es exactamente lo que no podía ocurrir en las prácticas 3, 4 y 5.

Captura N.º 8: consola de administración mostrando la cola notificaciones.correo con un mensaje en la columna Ready y ningún consumidor. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 3: La API respondió 201 aunque nadie haya enviado todavía el correo de bienvenida. ¿De qué trabajo se liberó el servicio de Inscripciones y qué promesa está haciendo a cambio al estudiante que se inscribió? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 3 — El consumidor y la demostración del desacoplamiento

Ahora el segundo microservicio. Y sobre todo: la prueba de que los dos ya no dependen el uno del otro para estar vivos.

Paso 1. Prepare el proyecto del consumidor

```
$ cd practica6-rabbitmq
$ mkdir notificaciones && cd notificaciones
$ npm init -y
$ npm install amqplib dotenv
# el consumidor no necesita express: no atiende peticiones HTTP
```

Copie el .env: Cree también aquí un archivo .env con la misma RABBITMQ_URL y con COLA=notificaciones.correo. El nombre de la cola debe coincidir exactamente con el del binding: la causa más común de «el consumidor no recibe nada» es una letra distinta.

Paso 2. Escriba el trabajo que hace el consumidor

Cree index.js en la carpeta notificaciones. El envío del correo se simula con una espera de un segundo y medio: interesa que el trabajo tarde, porque así se ven las colas.

```
notificaciones/index.js
require("dotenv").config();
const amqp = require("amqplib");
const COLA = process.env.COLA;

async function enviarCorreo(evento) {
  console.log("[correo] enviando a", evento.correo, "por", evento.id);
  await new Promise((r) => setTimeout(r, 1500));
  console.log("[correo] enviado a", evento.correo);
}
```

Paso 3. Escriba el bucle de consumo

Agregue el resto del archivo. Fíjese en las tres decisiones marcadas: cola durable, un mensaje a la vez y confirmación manual.

```
notificaciones/index.js (agregar al final)
async function main() {
```

```
const conexion = await amqp.connect(process.env.RABBITMQ_URL);
const canal = await conexion.createChannel();
await canal.assertQueue(COLA, { durable: true });
canal.prefetch(1); // un mensaje sin confirmar a la vez
console.log("[correo] esperando mensajes en", COLA);
canal.consume(COLA, async (mensaje) => {
  if (mensaje === null) return;
  const evento = JSON.parse(mensaje.content.toString());
  await enviarCorreo(evento);
  canal.ack(mensaje); // recién aquí el broker lo borra
}, { noAck: false });

main().catch((e) => { console.error(e.message); process.exit(1); });
```

Paso 4. Encienda el consumidor

Arránquelo en una cuarta terminal. Lo primero que hará es procesar el mensaje que quedó esperando desde el laboratorio anterior.

```
# terminal 4
$ node index.js
```

Publique dos inscripciones más con curl y observe cómo el consumidor las procesa una tras otra, con su segundo y medio de trabajo cada una.

Captura N.º 9: terminal del consumidor procesando los eventos, con el registro de cada correo enviado y su identificador de inscripción. → *péguela en la Hoja de Evidencias.*

Paso 5. Apague el consumidor y siga inscribiendo

Este es el paso central de la práctica. Detenga el consumidor con Ctrl+C. NO detenga el productor. Publique cinco inscripciones seguidas.

```
# terminal 4: detenga el consumidor con Ctrl+C

# terminal 3: cinco inscripciones con el consumidor apagado
$ echo '{"estudiante":"Alumno","correo":"a@usfx.bo","curso":"COM-600"}' > cuerpo.json
$ for i in 1 2 3 4 5; do curl -s -X POST -d @cuerpo.json \
  -H "Content-Type: application/json" \
  http://localhost:3000/inscripciones > /dev/null; echo "enviada $i"; done
```

La API siguió respondiendo 201 a las cinco. Nadie se enteró de que el servicio de Notificaciones no existe en este momento.

Captura N.º 10: consola de administración con la cola notificaciones.correo mostrando 5 mensajes en Ready, cero consumidores, y el gráfico de la cola creciendo. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 4: Con REST, si el servicio de Notificaciones estaba caído, la inscripción fallaba o se quedaba esperando. Aquí no falló nada. ¿Dónde está en este momento el trabajo pendiente, qué recurso está consumiendo mientras espera y qué ocurriría si el consumidor no volviera nunca? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 6. Encienda el consumidor y vea drenar la cola

```
$ node index.js
```

El consumidor recupera los cinco mensajes en el orden en que se publicaron y la cola vuelve a cero. Nada se perdió mientras estuvo apagado.

Captura N.º 11: la terminal del consumidor procesando los cinco mensajes acumulados y la cola de la consola bajando a cero en el gráfico. → *péguela en la Hoja de Evidencias.*

Acaba de demostrar el desacoplamiento en el tiempo: productor y consumidor ya no necesitan estar vivos a la vez. Ese es el cambio de fondo respecto de todo el bloque de comunicación anterior.

Laboratorio 4 — Exchange directo frente a exchange temático

El exchange directo compara la clave completa, letra por letra. El temático la compara por partes separadas con puntos, y permite comodines. Con eso alcanza para que un mismo evento alimente a tres áreas distintas sin que el productor sepa que existen.

Paso 1. Declare la topología por código

Cree el archivo `topologia.js` en la raíz de `practica6-rabbitmq`. Este script se ejecuta una sola vez y deja el broker preparado; en un proyecto real vive junto al despliegue.

```
practica6-rabbitmq/topologia.js
const amqp = require("amqplib");

const URL = "amqp://admin:admin123@localhost:5672";
const EX = "eventos.academicos";

// cola -> patron de clave de enrutamiento
const enlaces = [
  ["correo.bienvenida", "inscripcion.*"],
  ["finanzas.cobros", "pago.*"],
  ["auditoria.todo", "#"],
];
```

Paso 2. Complete y ejecute el script

```
practica6-rabbitmq/topologia.js (agregar al final)
async function main() {
  const conexion = await amqp.connect(URL);
  const canal = await conexion.createChannel();
  await canal.assertExchange(EX, "topic", { durable: true });
  for (const [cola, patron] of enlaces) {
    await canal.assertQueue(colas, { durable: true });
    await canal.bindQueue(colas, EX, patron);
    console.log("enlazada", colas, "con el patron", patron);
  }
  await conexion.close();
}

main();
```

Ejécute con `node topologia.js` desde la carpeta `practica6-rabbitmq`. Necesita `amqplib`: si el script no lo encuentra, ejécute desde una carpeta donde esté instalado o instálelo también en la raíz.

Captura N.º 12: página del exchange `eventos.academicos` en la consola, mostrando sus tres bindings con los patrones `inscripcion.*`, `pago.*` y `#`. → *péguela en la Hoja de Evidencias.*

Paso 3. Publique cuatro claves distintas y observe el reparto

Desde la consola, en el exchange eventos.academicos, use Publish message cuatro veces cambiando solo la clave de enrutamiento. Use un contenido cualquiera, por ejemplo {"n":1}.

```
# claves a publicar, una por vez, desde la consola:
inscripcion.confirmada
inscripcion.anulada
pago.registrado
certificado.emitido
```

Vaya a Queues and Streams y anote cuántos mensajes recibió cada cola. Antes de mirar, prediga el resultado en un papel: el ejercicio pierde valor si mira primero.

Captura N.º 13: lista de colas de la consola con los contadores de correo.bienvenida, finanzas.cobros y auditoria.todo después de publicar las cuatro claves. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 5: La clave certificado.emitido llegó solo a una de las tres colas. Explique por qué, y diga qué diferencia hay entre el comodín * y el comodín # con un ejemplo propio de su proyecto integrador. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 4. Cambie el productor a temático sin tocar al consumidor

En el .env del productor cambie EXCHANGE=eventos.academicos, y en cola.js cambie el tipo "direct" por "topic". Enlace la cola notificaciones.correo al nuevo exchange con el patrón inscripcion.* desde la consola. Reinicie el productor y publique una inscripción.

El consumidor no cambió ni una línea y sigue trabajando. El enrutamiento es responsabilidad del broker, no de los servicios.

Regla práctica: Una clave de enrutamiento se lee de lo general a lo particular: dominio.evento, o dominio.entidad.evento. Elegirla bien hoy es lo que permite agregar consumidores mañana sin renegociar nada con el productor.

Laboratorio 5 — Confirmaciones y durabilidad: qué se pierde y qué se salva

Un mensaje puede perderse en tres momentos distintos: al publicarlo, mientras espera en la cola y mientras se procesa. Cada uno tiene su defensa, y las tres son configuración, no código complicado.

Paso 1. Simule una caída durante el procesamiento

El consumidor tarda un segundo y medio en cada correo. Publique una inscripción y, mientras el consumidor está en medio del envío, deténgalo con Ctrl+C antes de que imprima «enviado».

```
# terminal 3
$ curl -s -X POST -d @cuerpo.json \
  -H "Content-Type: application/json" \
  http://localhost:3000/inscripciones

# terminal 4: Ctrl+C mientras dice "enviando..."
```

Mire la consola: el mensaje no se perdió. Estaba en Unacked y volvió a Ready en cuanto el broker detectó que la conexión se cerró sin confirmación. Vuelva a encender el consumidor y verá que lo procesa de nuevo, entero.

Captura N.º 14: consola mostrando la cola con el mensaje que pasó de Unacked a Ready tras la caída del consumidor. → *péguela en la Hoja de Evidencias.*

Paso 2. Compruebe qué ocurre sin confirmación manual

Cambie en el consumidor { noAck: false } por { noAck: true } y quite la línea canal.ack(mensaje). Repita el paso anterior: publique y corte el consumidor a mitad del trabajo.

```
notificaciones/index.js (cambio temporal)
}, { noAck: true }); // el broker borra el mensaje al entregarlo
```

El mensaje desapareció. El broker lo dio por entregado en el momento de mandarlo por la red y ya no tiene copia. El estudiante nunca recibirá su correo y en ninguna parte queda registro del hecho. Revierta el cambio antes de continuar.

Regla: noAck: true solo se justifica cuando perder un mensaje no le cuesta nada a nadie: telemetría, posiciones de un mapa en vivo, métricas que se vuelven a emitir en un segundo. Para un evento de dominio, nunca.

Captura N.º 15: consola mostrando la cola en cero después de la caída del consumidor con noAck en true: el mensaje se perdió. → *péguela en la Hoja de Evidencias.*

Paso 3. Distinga cola durable de mensaje persistente

Cree desde la consola una cola llamada prueba.volatil con Durability = Transient y enlázela al exchange inscripciones con la clave inscripcion.confirmada. Publique dos inscripciones: entrarán tanto en la cola durable como en la volátil. Detenga el consumidor para que los mensajes se queden esperando y reinicie el broker.

```
$ docker compose restart rabbitmq
# espere de nuevo el "Server startup complete" antes de mirar
$ docker compose logs rabbitmq | tail -3
```

Tras el reinicio, la cola prueba.volatil ya no existe: se fue con su contenido. La cola notificaciones.correo sigue ahí y sus mensajes también, porque la cola es durable y el productor los publicó con persistent: true. Si faltara cualquiera de las dos condiciones, el mensaje no sobreviviría.

Captura N.º 16: lista de colas después del reinicio del broker: la cola durable con sus mensajes intactos y la cola transitoria desaparecida. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 6: Se necesitan dos condiciones para que un mensaje sobreviva al reinicio del broker: la cola durable y el mensaje persistente. Explique qué pasa exactamente si se cumple solo una de las dos, y por qué la durabilidad tiene un costo que no siempre conviene pagar. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Confirmación manual, cola durable y mensaje persistente son tres interruptores independientes. Encenderlos todos cuesta rendimiento; apagarlos cuesta datos. La decisión es del diseñador, no del broker.

Laboratorio 6 — Reintentos limitados y cola de mensajes muertos

Hay mensajes que fallan una vez por mala suerte y hay mensajes que fallarán siempre, porque vienen mal formados. Reencolar sin límite convierte al segundo caso en un bucle infinito que consume el broker. La solución es intentar un número acotado de veces y apartar lo que no se puede procesar.

Paso 1. Borre la cola anterior antes de redeclararla

La cola `notificaciones.correo` va a recibir ahora un argumento nuevo. Una cola existente no se puede modificar: hay que borrarla. En la consola, entre a la cola y use `Delete queue`.

El error que verá si no lo hace: Si intenta declarar una cola que ya existe con argumentos distintos, el broker cierra el canal con `PRECONDITION_FAILED - inequivalent arg`. No es un error de su código: es el broker defendiendo una cola que quizá tenga mensajes de otro servicio. Borrar y volver a declarar es la salida en laboratorio; en producción se crea una cola nueva y se migra.

Paso 2. Declare el circuito de reintento y la cola muerta

Cree el archivo `dlq.js`. Son tres piezas: la cola principal, que manda lo rechazado al exchange de reintentos; la cola de reintento, que retiene cinco segundos y devuelve el mensaje a la principal; y la cola muerta, donde termina lo que agotó sus oportunidades.

```
practica6-rabbitmq/dlq.js (fragmento central)
await canal.assertExchange("inscripciones", "direct", { durable: true });
await canal.assertExchange("reintentos", "direct", { durable: true });

// principal: lo que el consumidor rechaza se va al exchange de reintentos
await canal.assertQueue("notificaciones.correo", {
  durable: true,
  deadLetterExchange: "reintentos",
});
await canal.bindQueue("notificaciones.correo", "inscripciones",
  "inscripcion.confirmada");
```

Paso 3. Complete la cola de espera y la cola muerta

```
practica6-rabbitmq/dlq.js (agregar al final)
// reintento: retiene 5 segundos y devuelve el mensaje a la cola principal
await canal.assertQueue("notificaciones.reintento", {
  durable: true,
  messageTtl: 5000,
  deadLetterExchange: "inscripciones",
  deadLetterRoutingKey: "inscripcion.confirmada",
});
await canal.bindQueue("notificaciones.reintento", "reintentos",
  "inscripcion.confirmada");

// muerta: nadie la consume automáticamente, la revisa una persona
await canal.assertQueue("notificaciones.muertos", { durable: true });
```

Ejecute `node dlq.js`. En la consola deben aparecer las tres colas; fíjese en las etiquetas `DLX` y `TTL` que la consola muestra junto a los nombres.

Paso 4. Haga que el consumidor cuente sus intentos

Cada vez que un mensaje muere y renace, el broker le agrega el encabezado `x-death` con el número de vueltas que lleva. Declare arriba del archivo la constante `LIMITE` con valor 3 y reemplace el cuerpo del consume por este:

```
notificaciones/index.js (reemplazar el consume)
canal.consume(COLA, (mensaje) => {
  const muertes = mensaje.properties.headers["x-death"];
  const intentos = muertes ? muertes[0].count : 0;
  if (intentos >= LIMITE) {
    console.error("[correo] sin mas intentos: a la cola muerta");
    canal.sendToQueue("notificaciones.muertos", mensaje.content,
```

```
        { persistent: true });  
    return canal.ack(mensaje);  
  }  
  try {  
    procesar(JSON.parse(mensaje.content.toString()));  
    canal.ack(mensaje);  
  } catch (e) {  
    console.error("[correo] intento", intentos + 1, "fallido:", e.message);  
    canal.nack(mensaje, false, false); // false: no reencolar aquí  
  }  
}, { noAck: false });
```

El tercer parámetro: En `canal.nack(mensaje, false, false)` el último argumento es `requeue`. Ponerlo en `true` devuelve el mensaje al frente de la misma cola, y con un mensaje que siempre falla eso es un bucle cerrado a toda velocidad. En `false`, el mensaje sale por el camino que definió `deadLetterExchange`, que es justamente lo que acabamos de construir.

Paso 5. Provoque el fallo repetido

Agregue al consumidor una función `procesar` que rechace los correos mal formados, y publique una inscripción con un correo sin arroba.

```
notificaciones/index.js (agregar la validación)  
function procesar(evento) {  
  if (!evento.correo || !evento.correo.includes("@")) {  
    throw new Error("correo invalido: " + evento.correo);  
  }  
  console.log("[correo] bienvenida enviada a", evento.correo);  
}
```

Publique con `curl` un cuerpo cuyo correo sea `"ana.punto.usfx.bo"`. El consumidor fallará, el mensaje irá a `notificaciones.reintento`, esperará cinco segundos, volverá, fallará otra vez. Observe el ciclo en la consola durante veinte segundos: el mensaje aparece y desaparece de una cola a la otra.

Captura N.º 17: terminal del consumidor mostrando los tres intentos numerados y fallidos del mismo mensaje, con cinco segundos entre uno y otro. → *péguela en la Hoja de Evidencias*.

Paso 6. Encuentre el mensaje en la cola muerta

Después del tercer intento, vaya a la cola `notificaciones.muertos` y recupérela con `Get messages` en modo «Nack message requeue true».

Ahí está el mensaje completo, con su contenido original. En un sistema real esta cola tiene una alarma: si le entra algo, alguien debe mirarlo.

Captura N.º 18: consola mostrando la cola `notificaciones.muertos` con el mensaje dentro y su contenido desplegado en `Get messages`. → *péguela en la Hoja de Evidencias*.

Pregunta N.º 7: La cola de mensajes muertos no tiene ningún consumidor automático conectado. ¿Por qué sería un error ponerle uno que reintente sin más? Describa qué debería hacer, paso a paso, la persona o el proceso que atienda esa cola. → *respóndala en la Hoja de Evidencias, con sus propias palabras*.

Laboratorio 7 — Idempotencia: el mismo evento dos veces sin doble efecto

RabbitMQ promete entregar cada mensaje al menos una vez, y esa promesa incluye el caso en que un mensaje ya procesado se entregue de nuevo porque el ack se perdió. El broker no puede evitarlo. El consumidor sí.

Paso 1. Dé al evento un identificador estable

El identificador debe nacer con el hecho, no con el envío. Si se genera al publicar, dos publicaciones del mismo hecho tendrán identificadores distintos y no habrá forma de reconocerlas. En el productor, use el id de la inscripción como identificador del evento.

```
inscripciones/cola.js (dentro de publicar)
return canal.publish(process.env.EXCHANGE, clave, cuerpo, {
  persistent: true,
  contentType: "application/json",
  messageId: evento.id, // el id del hecho, no del envío
});
```

Paso 2. Agregue el efecto que no se puede repetir

Hasta ahora el consumidor solo imprimía. Agréguele un efecto contable: cada evento procesado descuenta un cupo del curso. Un cupo descontado dos veces es un error que el estudiante ve.

```
notificaciones/index.js (agregar arriba)
const cupos = { "COM-600": 30 };

function descontarCupo(evento) {
  cupos[evento.curso] = (cupos[evento.curso] ?? 30) - 1;
  console.log("[cupos]", evento.curso, "quedan", cupos[evento.curso]);
}
```

Llame a `descontarCupo(evento)` dentro de `procesar`, después de la validación del correo.

Paso 3. Entregue el mismo evento dos veces

Simule la reentrega. Desde la consola, en la cola `notificaciones.correo`, recupere un mensaje con `Get messages` en modo «`Nack message requeue true`»: el mensaje vuelve a la cola y el consumidor lo procesa otra vez. También puede publicar dos veces el mismo cuerpo desde el exchange con la misma clave.

El cupo se descontó dos veces por una sola inscripción. Nadie se equivocó al programar: el mensaje llegó dos veces y el consumidor obedeció las dos.

Captura N.º 19: terminal del consumidor mostrando el mismo identificador de evento procesado dos veces y el cupo descontado dos veces. → *péguela en la Hoja de Evidencias.*

Paso 4. Haga idempotente al consumidor

Un consumidor idempotente registra lo que ya hizo y, ante un identificador conocido, confirma sin volver a actuar. Confirmar es importante: si no lo hiciera, el mensaje quedaría dando vueltas para siempre.

```
notificaciones/index.js (dentro del consume)
const procesados = new Set(); // en produccion: tabla con clave unica

const id = mensaje.properties.messageId;
if (procesados.has(id)) {
  console.log("[idempotencia] evento repetido", id, "- se ignora");
  return canal.ack(mensaje); // confirmar igual: ya esta hecho
}
procesar(JSON.parse(mensaje.content.toString()));
procesados.add(id);
```

```
canal.ack(mensaje);
```

Por qué un Set no alcanza: Un conjunto en memoria se borra cuando el consumidor se reinicia, y no lo comparten dos copias del mismo servicio corriendo a la vez. En un sistema real el registro es una tabla con restricción de unicidad sobre el identificador del evento, escrita en la misma transacción que el efecto. Aquí usamos el Set porque cabe en la pantalla; la idea es la misma.

Paso 5. Demuestre que ya no hay doble efecto

Reinicie el consumidor y repita la reentrega del paso anterior con el mismo mensaje.

El segundo intento se reconoce, se ignora y se confirma. El cupo se descuenta una sola vez.

Captura N.º 20: terminal del consumidor mostrando el mensaje repetido reconocido e ignorado, con el cupo descontado una sola vez. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 8: El broker garantiza la entrega al menos una vez, pero no exactamente una vez. Explique por qué le resulta imposible garantizarlo y en qué parte del sistema queda entonces la responsabilidad. Dé un ejemplo de su proyecto integrador donde una entrega repetida causaría un daño visible al usuario. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Con esto se cierra el bloque de comunicación de la materia: sabe llamar a un servicio y esperar la respuesta, y sabe anunciar un hecho y seguir trabajando. Elegir entre las dos cosas, con criterio, es lo que se evalúa de aquí en adelante.

PARTE 2 — EJERCICIOS PROPUESTOS

Esta parte se resuelve de forma individual. Cada ejercicio se entrega con: (a) el código o los archivos de configuración en el repositorio, (b) las capturas que demuestren su funcionamiento y (c) la explicación pedida, en el informe PDF.

Nota: Los seis ejercicios se resuelven sobre SU proyecto integrador, no sobre el ejemplo de inscripciones. Elija un hecho real de su dominio que hoy resuelva de forma síncrona —una venta confirmada, un pago acreditado, una orden despachada— y conviértalo en un evento. Del 1 al 4 son obligatorios para todos; el 5 y el 6 son los que distinguen una entrega correcta de una sobresaliente.

Ejercicio 1. Un evento de dominio propio (10 pts)

Identifique en su proyecto integrador una operación que hoy dispare trabajo adicional de forma síncrona (enviar un correo, generar un comprobante, actualizar un tablero) y publique un evento en el momento en que esa operación se confirma.

Se pide:

- Un exchange temático propio, con un nombre que describa el dominio y no la tecnología.
- Una clave de enrutamiento jerárquica documentada en el README, con la regla que usó para construirla.
- El evento publicado con `persistent: true` y con un identificador estable en `messageId`.
- La publicación ocurre después de que la operación quedó confirmada, nunca antes; explique en el informe qué pasaría si fuera al revés.
- Captura de la consola mostrando el exchange con sus bindings.

Criterio de aceptación: *El evento debe nombrar un hecho ocurrido en pasado (PagoAcreditado), no una orden a otro servicio (EnviarCorreo). Un evento que ordena es una llamada síncrona disfrazada.*

Ejercicio 2. Desacoplamiento demostrado (8 pts)

Escriba un segundo microservicio que consuma ese evento y demuestre con evidencia que ambos servicios ya no dependen el uno del otro para estar vivos.

Se pide:

- Consumidor independiente, con su propio package.json y su propia configuración.
- Secuencia demostrada en capturas: consumidor apagado, cinco operaciones aceptadas por la API, cola acumulando, consumidor encendido, cola en cero.
- El tiempo de respuesta de su API medido con el consumidor encendido y con el consumidor apagado, con los números en el informe.
- Una explicación de qué habría ocurrido en esa misma secuencia si la comunicación fuera REST, como en la Práctica 3.

Criterio de aceptación: *Ninguna de las cinco operaciones puede haber fallado ni haberse demorado por la ausencia del consumidor.*

Ejercicio 3. Enrutamiento por tema hacia tres consumidores (12 pts)

Su evento debe alimentar a tres colas distintas con intereses distintos, sin que el productor sepa cuántos consumidores existen.

Se pide:

- Tres colas enlazadas con patrones distintos: uno específico, uno con comodín de una palabra y uno que reciba todo el dominio.
- Un script de topología ejecutable que cree exchanges, colas y bindings desde cero en un broker vacío.
- Una tabla en el informe con al menos cinco claves de enrutamiento y una marca por cada cola que las recibe.
- La demostración de que agregar la tercera cola no requirió modificar ni reiniciar el productor.
- Captura de la consola con los contadores de las tres colas después de publicar las cinco claves.

Criterio de aceptación: *La tabla debe estar predicha antes de ejecutar y comparada después con lo observado. Si alguna predicción falló, explique por qué: ahí está el aprendizaje.*

Ejercicio 4. Ningún mensaje perdido (10 pts)

Configure su circuito para que ningún evento se pierda ante la caída del consumidor ni ante el reinicio del broker, y demuestre cada garantía por separado.

Se pide:

- Confirmación manual con prefetch acotado y justificación del valor elegido.
- Evidencia de la caída a mitad de proceso: el mensaje pasa de Unacked a Ready y se procesa completo al reconectar.
- Evidencia del reinicio del broker con cola durable y mensaje persistente: los mensajes siguen ahí.
- La contraprueba: la misma situación sin durabilidad o sin persistencia, con el mensaje perdido y la captura que lo demuestra.

Criterio de aceptación: *Se exigen las dos caras: la garantía funcionando y la misma garantía apagada fallando. Una sola de las dos no acredita el ejercicio.*

Ejercicio 5. Cola de mensajes muertos con reintentos acotados (10 pts)

Construya el circuito completo de reintentos para su consumidor: un mensaje que falla se reintenta un número limitado de veces con espera entre intentos y, agotadas las oportunidades, se aparta en una cola de mensajes muertos.

Se pide:

- Cola principal con exchange de mensajes muertos declarado, cola de reintento con tiempo de vida y cola muerta final.
- Un límite de intentos leído del encabezado x-death, no un contador en memoria del consumidor.
- Un mensaje envenenado provocado a propósito, con la captura de los intentos sucesivos y su llegada a la cola muerta.
- El contenido del mensaje muerto recuperado desde la consola, con la explicación de qué información necesitaría un operador para decidir qué hacer con él.
- Un procedimiento escrito, de cinco líneas como máximo, para atender esa cola.

Criterio de aceptación: *El mensaje debe fallar varias veces antes de morir, con espera entre intentos. Un mensaje que va a la cola muerta al primer fallo no cumple el ejercicio.*

Ejercicio 6. Consumidor idempotente demostrado (10 pts)

Su consumidor debe producir un efecto que no se pueda repetir sin daño —descontar un cupo, acreditar un pago, emitir un comprobante— y debe resistir la entrega repetida del mismo evento.

Se pide:

- La demostración del daño primero: el mismo evento entregado dos veces y el efecto duplicado, con la captura.
- La corrección después: registro de eventos procesados con el identificador del evento y confirmación del duplicado sin repetir el efecto.
- El registro fuera de la memoria del proceso: una tabla, un archivo o una colección que sobreviva al reinicio del consumidor.
- La prueba de que sobrevive: reinicie el consumidor entre la primera y la segunda entrega y muestre que sigue reconociendo el duplicado.
- Una justificación de por qué el identificador que eligió es estable y no cambia entre dos entregas del mismo hecho.

Criterio de aceptación: *Se evalúa la secuencia completa: daño demostrado, corrección aplicada y duplicado ignorado tras reiniciar el consumidor. Sin la evidencia del daño previo, el ejercicio no acredita.*

Rúbrica de evaluación

Parte 1 — Práctica guiada (40 puntos)

Criterio	Descripción	Pts
Capturas completas	Las 20 capturas están pegadas en la Hoja de Evidencias, son legibles y corresponden a la máquina del estudiante (se ve su usuario o el nombre del equipo).	15
Ejecución correcta	Las salidas muestran el resultado esperado, sin errores sin resolver.	10
Preguntas de comprobación	Las 8 preguntas están respondidas con criterio propio, no copiadas del enunciado.	10
Orden y presentación	Hoja de Evidencias completa, con datos del estudiante y declaración firmada, entregada en PDF y dentro del plazo.	5
Subtotal		40

Parte 2 — Ejercicios propuestos (60 puntos)

Ejercicio	Qué se evalúa	Pts
1. Un evento de dominio propio	Evento de dominio bien nombrado, clave jerárquica justificada, publicación persistente con identificador estable y en el momento correcto.	10

Ejercicio	Qué se evalúa	Pts
2. Desacoplamiento demostrado	Consumidor independiente, secuencia completa de desacoplamiento evidenciada, medición de tiempos y comparación argumentada con el caso síncrono.	8
3. Enrutamiento por tema hacia tres consumidores	Tres patrones bien diferenciados, script de topología reproducible, tabla de enrutamiento predicha y verificada, productor intacto.	12
4. Ningún mensaje perdido	Confirmación manual justificada, ambas garantías demostradas y sus contrapruebas correspondientes.	10
5. Cola de mensajes muertos con reintentos acotados	Circuito de tres colas correcto, límite basado en x-death, mensaje envenenado demostrado hasta la cola muerta y procedimiento de atención escrito.	10
6. Consumidor idempotente demostrado	Efecto no repetible identificado, doble efecto demostrado, registro persistente de procesados, duplicado ignorado tras reinicio e identificador justificado.	10
Subtotal		60

Penalizaciones

Situación	Descuento
Consumidor con noAck en true sobre un evento de dominio, sin justificación	-5 pts
Reencolar sin límite un mensaje que siempre falla (nack con requeue en true y sin cola muerta)	-5 pts
Credenciales del broker escritas dentro del código en lugar del archivo de configuración	-3 pts
node_modules subido al repositorio (falta .gitignore)	-3 pts
Eventos nombrados como órdenes (EnviarCorreo en lugar de InscripcionConfirmada)	-3 pts
Capturas que no corresponden a la máquina del estudiante	Anulación de la Parte 1
Entrega fuera de plazo	Según reglamento de la materia

Anexo — Errores frecuentes y cómo resolverlos

Mensaje o síntoma	Causa probable	Solución
connect ECONNREFUSED 127.0.0.1:5672	El broker no está corriendo todavía, o arrancó hace menos de quince segundos.	Ejecute docker compose ps y espere el mensaje «Server startup complete» en los registros antes de arrancar los servicios.
Handshake terminated by server: 403 ACCESS_REFUSED	Usuario o contraseña incorrectos, o el usuario guest usado desde fuera de la máquina del broker.	Revise la URL del archivo .env: debe llevar el usuario admin creado en el docker-compose.yml.
PRECONDITION_FAILED - inequivalent arg durable / x-dead-letter-exchange	Se está declarando una cola que ya existe con argumentos distintos.	Borre la cola desde la consola y vuelva a ejecutar el script de topología, o declare una cola nueva con otro nombre.
El consumidor arranca pero nunca recibe nada	La cola no está enlazada al exchange, la clave de enrutamiento no coincide o el nombre de la cola difiere en una letra.	Abra el exchange en la consola y verifique el binding; publique a mano desde la consola con la misma clave para aislar el problema.
Los mensajes desaparecen y el trabajo nunca se completa	El consumidor usa noAck en true, o hace ack antes de terminar el trabajo.	Ponga noAck en false y llame a ack solo después de que el procesamiento terminó bien.
El mismo evento se procesa dos veces	Reentrega tras una caída, un ack perdido o una publicación repetida. El broker entrega al menos una vez.	Haga idempotente al consumidor: registre el identificador del evento y confirme sin repetir el efecto si ya está.

Mensaje o síntoma	Causa probable	Solución
La cola crece y nunca baja	No hay consumidores conectados, o el consumidor procesa más lento de lo que el productor publica.	Verifique la columna Consumers en la consola; si hay consumidor, levante más copias o suba el prefetch tras medir.
Un mensaje da vueltas sin parar entre la cola y el consumidor	El consumidor rechaza con requeue en true un mensaje que siempre falla.	Use nack con requeue en false y un exchange de mensajes muertos, con un límite de intentos leído de x-death.

Antes de pedir ayuda: Antes de pedir ayuda, abra la consola de administración y responda tres preguntas: ¿la cola tiene consumidores conectados?, ¿el exchange tiene el binding con la clave que usted está publicando?, ¿los mensajes están en Ready o en Unacked? Esas tres respuestas identifican casi todos los problemas de esta práctica sin tocar el código.

Referencias

- RabbitMQ Tutorials para Node.js — <https://www.rabbitmq.com/tutorials>
- amqp-lib, referencia de la API de canales — https://amqp-node.github.io/amqp-lib/channel_api.html
- RabbitMQ, complemento de administración — <https://www.rabbitmq.com/docs/management>
- RabbitMQ, colas de mensajes muertos — <https://www.rabbitmq.com/docs/dlx>
- Richardson, Chris. Microservices Patterns, capítulo 3: comunicación entre procesos. Manning, 2018.
- Hohpe, Gregor y Woolf, Bobby. Enterprise Integration Patterns. Addison-Wesley, 2003.
- Diapositivas del Tema 4 — Comunicación entre Microservicios (COM-600).